Assessment 1: AI-Powered CV Builder

Student name: Hyunbeen Suk **Student ID:** 14655765 **Word count:** ~1,510 (Sections 1–5 prose; tables, appendices and code listings excluded)

Declaration of GenAI use. A coding agent (Claude Code) was used to plan, build, debug and test the system; each prompt, generated change and verification is listed in `prompt_record.md` (in `A1_Code_14655765.zip`). This report was first drafted with an LLM from the Report Prompt and the supplied materials folder, then every statement, number, file name and `document_id` was checked against the source code and the recorded evidence files and corrected where the draft was inaccurate (paraphrased retrieval excerpts, an unverified ranking claim and a generic agent trace were replaced with the recorded evidence).

1. Task and System

The **AI-Powered CV Builder** converts a user's plain-English introduction and follow-up answers into a confirmed profile, retrieves relevant job descriptions, generates a job-specific CV, and exports the result as a PDF. The main inputs are the user's introduction, the confirmed profile, and the supplied job records in `data/jobs.jsonl`. The main output is a tailored CV PDF; the editable CV content is held internally as a `CV` object.

The workflow begins in `cli.py`, where `bootstrap_profile()` in `intake.py` extracts only information explicitly stated in the introduction. A `ProfileAgent` then inspects the profile and may call `ask_user`, `update_profile`, `retrieve_jobs`, or `finish`. Confirmed information is stored in `ProfileState` (`profile.py`), which is kept separate from job-specific CV content, so one confirmed profile can drive several CVs.

The `JobIndex` in `retrieval.py` loads the nine job records, embeds them once, and returns the top three jobs with cosine scores and supporting excerpts. The user selects a job through the CLI. `generate_cv()` in `cv_generator.py` sends the confirmed profile and the selected job to the LLM and produces a summary, matched skills, missing requirements, and editable sections; a deterministic fact-check then removes any unsupported matched-skill claim. `pdf_export.render()` produces the polished A4 PDF. Section editing before export is implemented in `cli.py` (`_edit_loop`) and is shown in the demo video.

Each run writes the confirmed profile to `outputs/profile_<user>.json` and one PDF per selected job to `outputs/CV_<user>_<document_id>.pdf`, so the same run can build several CVs and a later run for the same user starts from the saved profile. Selecting a different target job re-invokes only `generate_cv()` and `pdf_export.render()`; the `ProfileState` is not touched, which is why the two-job CVs in Section 3.2 share an identical profile.

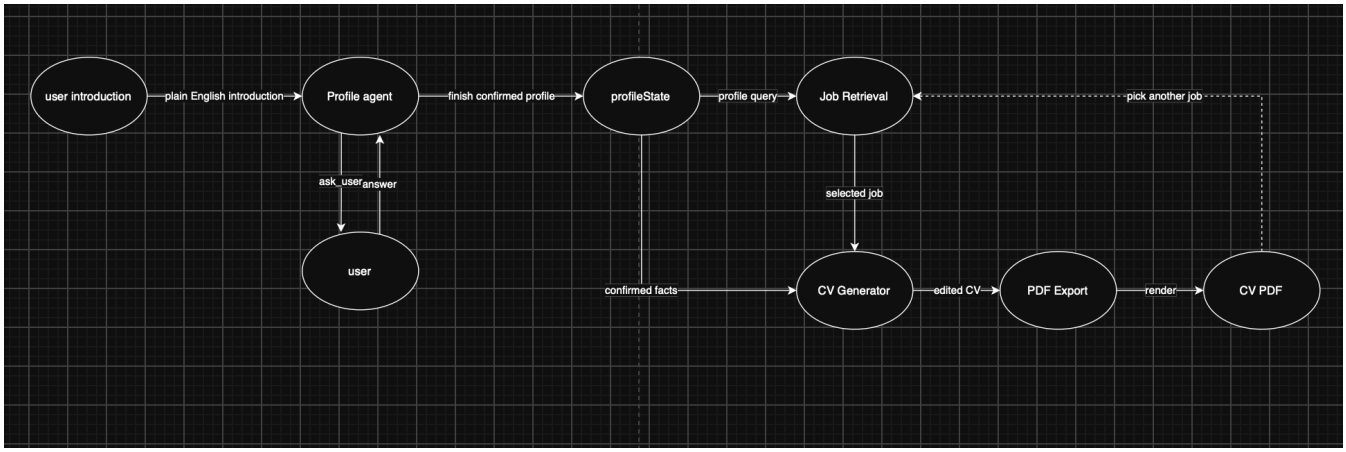


Figure 1. Data flow from the user introduction through `bootstrap_profile()`, `ProfileAgent`, `ProfileState`,

`JobIndex`, CV generation and `pdf_export.render()`.

2. Prompt, Agent and RAG

2.1 Prompt Design

The key prompt improvement changed the CV-generation prompt from `CV_SYSTEM_V1` to `CV_SYSTEM_V2` in `prompts.py`. The goal was to stop the model inventing skills or rewriting the candidate's experience, and to separate matched skills from missing job requirements. It was tested with `student_profile_01` (Maya Chen) and the `rag_agent_engineer` job.

`CV_SYSTEM_V1` told the model to *"Make the candidate look as strong as possible for the job."* Its output added `"AI Agents & Tool Calling"`, `"Prompt Design"` and `"JSON Data Handling"` to the skills list, although none appear in the confirmed profile, and it rewrote the retail experience into *"Collaborated within a fast-paced team environment to ... resolve operational challenges"*. It gave a single flat `skills` list with no matched-vs-missing distinction.

`CV_SYSTEM_V2` adds a role line (*"You do not evaluate or embellish"*), an evidence boundary — *"Allowed evidence: ONLY the confirmed profile for candidate claims; ONLY the job description for requirement/match reasons"* — and a fixed JSON schema requiring `profile_evidence` for each matched skill and `job_line` for each requirement. For the same input, V2 claimed only `Python`, linked to *"Team Course Planner and AI Coursework Prototype"*, and listed the master's degree and "three or more years" requirements separately as missing.

Because a prompt cannot be fully trusted, `cv_generator._supported()` then re-checks each matched skill's evidence by token overlap (≥ 0.5) against the profile corpus and drops entries that fail. Across the four sample CVs, `dropped_claims` was `[]` in every case. Seeing V1 fabricate skills is what motivated both the "Allowed evidence" clause and this deterministic check.

2.2 Tool-Calling Agent

`ProfileAgent` (`agent.py`, LLM `gemini-flash-lite-latest`) exposes four function-declaration tools:

Tool	Arguments	Function
`update_profile`	`fields` (object)	merges confirmed information into `ProfileState`; Observation = new `missing` list
`ask_user`	`question` (string)	asks one follow-up question; Observation = the user's answer
`retrieve_jobs`	`query` (string), `k` (int)	searches the job index; `k` is clamped to `top_k = 3`
`finish`	`reason` (string)	stops the loop

A representative Action→Observation cycle occurred at step 5 of the incomplete-profile run:

```
Action: update_profile {"fields": {
  "technical_skills": [{"skill": "Python", "evidence": "..."}],
  "projects": [{"title": "Library Catalogue", ...}, {"title": "Data Structures Assignments",
...}]}}
```

`ProfileAgent._run_tool()` calls `profile.merge()`; `_normalise_field()` coerces the malformed `technical_skills` item to the string `"Python"` and keeps the two project dictionaries. The Observation `profile updated. missing: ['experience', 'skill_evidence for: Python']` is returned to the model as a `function_response`, and the next Action is chosen from the updated state.

For the incomplete user the agent asked **three** useful questions (name, contact email, where Python was used), then retrieved jobs and called `finish`. Given a profile that is already complete (the recorded runs build the five profiles directly from `data/student_profiles.jsonl`) it makes **no** `ask_user` calls and goes straight to retrieval, then `finish`. The interactive CLI instead starts from the short introduction, so `bootstrap_profile()` leaves gaps and the agent does ask follow-ups, as the demo video shows. The stopping condition is `finish` or `max_steps = 8`; both recorded runs stopped with `finish`.

A limitation is visible in the incomplete-profile trace: the agent's `finish` reason claimed the profile was complete while `ProfileState.missing()` still reported `experience` and `skill_evidence for: Python`. The stopping mechanism works, but the model's stated reason was not consistent with the recorded state.

2.3 Retrieval-Augmented Generation

The RAG component spans `data_loader.py`, `embeddings.py` and `retrieval.py`. `load_jobs()` reads `data/jobs.jsonl` and checks that all nine records have unique `document_id` values. `job_text()` builds each job's embedding text as `"{title}. {title}. {description}"`, weighting the role name. Embeddings run locally with `sentence-transformers/all-MiniLM-L6-v2` (no API key), reached through an `EmbeddingBackend` protocol that also lets tests inject a deterministic fake. `JobIndex` embeds each job once and caches the matrix to `outputs/job_index.npz`. `profile_query()` builds the query from the job goal (repeated twice), up to eight technical skills, project titles and the summary. `search()` embeds the query, computes cosine similarity on the unit vectors, ranks with `argsort`, and attaches the highest-overlap description sentences via `_best_excerpt()`.

The top three for `student_profile_02` after the retrieval repair (verbatim excerpts from `retrieval_results.md`):

Rank	document_id	Score	Supporting excerpt
1	`graduate_ai_engineer`	0.648	"Requirements: A degree in artificial intelligence, data science, computer science ...; knowledge of machine-learning fundamentals, statistics, data analysis, agentic AI, LLMs, RAG ...; Python with NumPy, Pandas, scikit-learn, and PyTorch ..."
2	`rag_agent_engineer`	0.607	"Responsibilities: Develop LLM-powered AI applications; design and optimise RAG pipelines and AI-agent workflows; build AI-powered knowledge systems and retrieval solutions ..."
3	`nlp_multimodal_ml_engineer`	0.569	"Requirements: At least four years of industry AI or machine-learning experience, preferably in NLP or multimodal AI; a master's degree or higher; ... Python or Java; ML and deep-learning libraries; transformer architectures ..."

Retrieval therefore returns ranked records with evidence, not just identifiers. The five-profile test also shows the nine-job dataset is concentrated on AI/ML-engineering roles, which limits retrieval for other career areas.

3. Evaluation

3.1 Five-Profile Test

Five confirmed synthetic profiles were compared with predictions made before running retrieval.

Profile	Career goal	Expected relevant job(s)	Actual top-3	Result
01 Maya Chen	Software / backend	`rag_agent_engineer`, `graduate_ai_engineer`	`agentic_ai_engineer`, `rag_agent_engineer`, `graduate_ai_engineer`	Pass

02 Liam Patel	AI / NLP	`nlp_multimodal_ml_engineer`, `rag_agent_engineer`	`graduate_ai_engineer`, `rag_agent_engineer`, `nlp_multimodal_ml_engineer`	Pass
03 Sofia Williams	Data / BI	`graduate_ai_engineer`, `agentic_ai_data_scientist`	`graduate_ai_engineer`, `rag_agent_engineer`, `agentic_ai_data_scientist`	Pass
04 Noah Ibrahim	Cybersecurity / systems	`trust_safety_ml_engineer` (weak)	`agentic_ai_engineer`, `agentic_ai_data_scientist`, `rag_agent_engineer`	Fail
05 Ethan Nguyen	Backend / cloud	`agentic_ai_engineer`, `rag_agent_engineer`	`agentic_ai_engineer`, `agentic_ai_data_scientist`, `rag_agent_engineer`	Pass

The measured result was **4/5** predicted jobs in the top three. Profile 04 is not treated as a ranking error: the dataset contains no genuine security-engineering role.

3.2 Two Jobs for the Same User

The same confirmed `student_profile_01` was used for `agentic_ai_engineer` and `rag_agent_engineer`.

Aspect	`agentic_ai_engineer` CV	`rag_agent_engineer` CV
Summary emphasis	Python, REST APIs, the web API service	Python, AI coursework, the tool-call agent prototype
Section order	Education → Technical Skills → Projects → Experience → Achievements	same
Matched skills	`Python`	`Python`
Missing requirements	2+ years Python, Langflow, LLM/RAG/agent experience, cloud, Terraform/Kubernetes, Docker, stakeholder engagement	master's degree, 3+ years' experience, LLM/RAG/agent experience, LangChain/LangGraph/LlamaIndex, vector databases, cloud AI platforms

The confirmed profile is unchanged; only the job-specific content and the missing-requirements list move, each drawn from the respective job description. Neither CV dropped any claim.

3.3 Cross-User Test

`student_profile_01` and `student_profile_02` were run in sequence through the same `cv_builder` code path. Each user gets its own `ProfileState` (keyed by `user_id`, no shared global state) and its own `ProfileAgent`. The unit test `test_profile_isolation` mutates Maya's skills with `"INJECTED"` and asserts Liam's are unchanged (passes). The observed CVs contain different projects and skills; Maya's top three are `agentic_ai_engineer`, `rag_agent_engineer`, `graduate_ai_engineer`, while Liam's third result is `nlp_multimodal_ml_engineer`. No project or experience text crossed between users.

3.4 Failure and Improvement

The main observed failure was for Liam's NLP profile: before the repair, `nlp_multimodal_ml_engineer` ranked **below the top three** — the top three were `graduate_ai_engineer` (0.656), `rag_agent_engineer` (0.605) and `agentic_ai_engineer` (0.555). The likely cause was shared vocabulary: all nine jobs contain terms such as Python, Git, SQL, Docker, RAG and agents, and `profile_query()` listed the job goal only once. The repair repeated the job goal twice in the query, cut the skill list to eight items, and changed `job_text()` to `return f"{title}. {title}. {job.get('description', '')}".strip()`. The retest produced `graduate_ai_engineer` (0.648), `rag_agent_engineer` (0.607) and `nlp_multimodal_ml_engineer` (0.569), moving the NLP role into the top three. Re-running `student_profile_01` end to end afterwards showed no

regression: the profile JSON was unchanged, the agent still stopped with `finish`, and both CVs were produced with no dropped claims.

4. Vibe Coding Process

The Coding Agent (`Claude Code`, named in `prompt_record.md`) supported planning, implementation, debugging and testing. Development was iterative: each prompt was tied to a concrete problem and a verification method.

Development goal	Prompt / change	Test and decision
Project scaffold	*"Scaffold + plan"*	created the `cv_builder/` package and a config-switchable embedding backend; import and retrieval tests passed → accepted
Free LLM provider	*"구글에서 get an api ... anthropic 말고 다르게"*	replaced the Anthropic SDK with `google-genai`; a live end-to-end call worked → Gemini retained
Slow model calls	added a 90-second timeout, retry, and `thinking_budget=0`	timed calls fell from 54 s to 4 s → accepted
Malformed agent fields	added `_normalise_field()` and `test_merge_normalises_messy_model_output`	5/5 tests pass and the full run completes → accepted
Retrieval failure	reweighted the job goal and job title	re-running retrieval moved `nlp_multimodal_ml_engineer` into Liam's top three → accepted

The Prompt Record also shows constraints shaping implementation: every LLM response is cached in `outputs/llm_cache/` to stay within the free-tier quota, and the PDF exporter was repaired after an `FPDFUnicodeEncodingException`. Each change was judged against an observed problem rather than added as a planned feature. The next improvement identified is that the agent never asks about work `experience`.

5. Ethics and Limitations

Privacy. API credentials stay outside the report and source: `config.py` resolves the key from `GOOGLE_API_KEY`, and `config.example` holds only placeholders. All profile data is explicitly synthetic (`data/README.md`), and the submission contains no `config.yaml` or key.

Unsupported claims. Fact accuracy has two layers: `intake.py` instructs the extraction model to use only stated information, and `CV_SYSTEM_V2` limits candidate claims to the confirmed profile. `cv_generator._supported()` adds a deterministic check on matched-skill evidence. Its limit is that it only guards `matched_skills` — it does not independently verify every summary sentence or section item.

Bias / retrieval reliability. Retrieval is only as good as the job set. All nine jobs are AI/ML-engineering roles, so `student_profile_04` (security) receives unrelated AI roles and every profile clusters on a few postings; scores are close (0.52 vs 0.52), so ranking is sensitive to query wording, as Section 3.4 shows.

Agent / model failure. The agent can call `finish` while `missing()` is non-empty (the incomplete-user run ended still missing `experience`), so the stopping logic is not proof that a profile is complete. The CV is still generated from whatever is confirmed, and `missing()` records the gaps rather than hiding them.

Safeguards. Three checks keep the output inspectable rather than trusted blindly: `missing()` is printed after the agent stops, so gaps are visible before a CV is made; `_supported()` prints every dropped matched-skill claim; and each retrieved job is shown with its `document_id` and excerpt so a match reason can be traced to

the job text. Broader job coverage, stronger final-state validation and fuller fact-checking (covering summary and section text, not only matched skills) remain future work.

References

- `sentence-transformers` — `all-MiniLM-L6-v2` sentence-embedding model.
- Google `google-genai` SDK — model `gemini-flash-lite-latest`.
- `fpdf2` — PDF generation.

Appendices

Appendix A — Architecture Diagram

See Figure 1 (Section 1). Source image: `architecture_diagram.png`.

Appendix B — Selected Prompt Record (source: `prompt_record.md` (in `A1_Code_14655765.zip`))

V1 prompt:

```
You write tailored CVs. Given a candidate profile and a target job, produce a
CV in JSON with a summary, skills, experience, projects, and a list of missing
requirements. Make the candidate look as strong as possible for the job.
```

V2 principle:

```
Allowed evidence: ONLY the confirmed profile for candidate claims;
ONLY the job description for requirement/match reasons.
```

Appendix C — Agent Trace (source: `test_results/agent_actions_and_observations.md`)

Complete profile (`student_profile_01`):

```
Action (step 0): retrieve_jobs {"k": 5, "query": "Software Engineering Intern, ... | Python |
Java | SQL | REST APIs | Git | GitHub | pytest | Jupyter Notebook"}
Observation (step 0): top jobs:
  - agentic_ai_engineer   (score 0.521)
  - rag_agent_engineer    (score 0.520)
  - graduate_ai_engineer  (score 0.505)
Action (step 1): finish {"reason": "Profile complete and relevant jobs successfully retrieved."}
Observation (step 1): done
```

`k` was passed as 5 but clamped to `top_k = 3`. The incomplete-profile trace additionally records three `ask_user` actions (name, email, Python evidence) before `retrieve_jobs` and `finish`.

Appendix D — Retrieval Results (source: `test_results/retrieval_results.md`)

Backend `local:sentence-transformers/all-MiniLM-L6-v2`, `top_k = 3`. Repaired ranking for `student_profile_02`: `graduate_ai_engineer` 0.648, `rag_agent_engineer` 0.607, `nlp_multimodal_ml_engineer` 0.569, with verbatim excerpts (Section 2.3). All six profiles' queries and excerpts are in the source file.

Appendix E — Five-Profile Results (source: `test_results/five_profile_results.md`)

4/5 predicted jobs appeared in the top three; `student_profile_04` is an acknowledged dataset limitation.

Appendix F — Failure and Repair (source: `test_results/failure_and_repair.md`)

Failure: NLP role below the top three. Repair: `profile_query()` and `job_text()` reweighting. Retest: NLP role at rank 3. Repeat of `student_profile_01`: two CVs, no dropped claims, no regression.

Appendix G — Code Extracts

```
# retrieval.py — profile query weighting
parts: list[str] = [goal, goal] if goal else []
parts += profile.get("technical_skills", [])[:8]

# cv_generator.py — matched-skill fact check
for m in data.get("matched_skills", []):
    if _supported(m.get("profile_evidence", ""), corpus):
        kept.append(m)
    else:
        dropped.append(f"{m.get('skill')}: {m.get('profile_evidence')}")

# profile.py — completeness
def is_complete(self) -> bool:
    return not self.missing()
```

Verification Checklist

- [x] Every described feature exists in the source code.
- [x] Every reported test has an actual result (`pytest` 5/5; batch run outputs in `test_results/`).
- [x] Every Agent claim has an Action and Observation (`agent_actions_and_observations.md`, `follow_up_question_test.md`).
- [x] Every retrieval claim uses the correct `document_id` and verbatim excerpt (`retrieval_results.md`).
- [x] Different users remain separate (`test_profile_isolation`, separate profile JSON files).
- [x] Every failure has an honest explanation (Sections 2.2, 3.1, 3.4, 5).
- [x] Credentials stay outside the report (placeholders only).